Three Dimensional Protein Structure Alignment Algorithms with Selective Secondary Structure Information

Wei-De Jiang Yaw-Ling Lin* Hong-Shin Chen

Department of Computer Science and Information Engineering,
Providence University,
200 Chung Chi Road, Shalu, Taichung County, Taiwan 433.
winderek@gmail.com, yllin@pu.edu.tw, hongxin0118@gmail.com

Abstract. *The knowledge of structural classes is applied in numerous important predictive tasks that address structural and functional features of proteins. In our previous work [17], algorithms are proposed to refine the given alignment by stepwise finding better alignments of the protein pairings using minimum bipartite matching method on geometric distance space and several other adjustment strategies. The proposed methods are used to improve the given initialized alignment of two structures. The refinement method is applied to improve several previous known methods including VAST [7] and CE [18].*

In this paper, we propose methods of combining different segmentation methods that incorporate the secondary structural information. The experimental results show that our method improves the aligned rmsd *value obtained from the VAST package by an averaged improvement ratios about 18%. The best quality is obtained by combining different sources of secondary structural information. These settings are later adjusted by local refinement algorithm with trigonometric series estimation. Here we also propose ways of balancing two constraints of the time resource as well as the quality requirement. Systematic ways of selecting appropriate algorithms according to different users' quality/time preference are addressed. We show that it is possible to adjust the parameters to get the result that are sufficiently satisfactory and yet spent much less time resource. Furthermore, some of our preliminary result is accessible through the web interface.*

Keywords: structural proteomics, algorithms, structure alignments and comparisons, *rmsd*, minimum bipartite matching, secondary structure, combined algorithms.

1 Introduction

Sequence-derived structural and physicochemical features have frequently been used in the development of statistical learning models for predicting proteins and peptides of different structural, functional and interaction profiles. In the past few years, greedy local search strategies became popular. These algorithms incrementally alter inconsistent value assignments to all the variables. Information on the structural classes of proteins is useful for studying the broader problem of protein structure prediction and carrying out a number of predictive tasks that address certain structural and functional features [11]. In our previous work [17], we show that the trigonometric series approximation is appropriate for estimating the good isometric transformations of one structure and aligning it to the other structure.

Based on these results, here we propose algorithms to refine the given alignment by stepwise finding better alignments of the protein pairings using minimum bipartite matching method on geometric distance space and several other adjustment strategies. The proposed methods are used to improve the given initialized alignment of two structures. The refinement method is applied to improve several previous known methods including VAST [7] and CE [18]. Nevertheless, the quality of the final results obtained from the local search and refinement methods highly depend on the initial settings of starting search positions. Here in this paper, we propose several possible settings of the initialization algorithms to examine the effectiveness of the local refinement algorithms given different initial settings obtained from the secondary structural information of the given protein pair.

*Corresponding author. This work is supported in part by the National Science Council (NSC 97-2221-E-126-009), Taiwan, Republic of China.

2 Background and Terminology

The main idea of our local refinement algorithm for finding a suitable matching between two sets of points before utilizing the Rmsd procedure to fine-tune the final result is by adjusting the suitable parameter sets by ways of searching the underlying parametric space.

Root mean squared deviation

The smallest *root mean squared deviation (rmsd)* is a least-squares fitting method for two sequences of points [9]. The idea is to align atom vectors of the two given (molecular) structures, and use the common least averaged squared errors as a measurement of differences between these two (paired) sequences. Formally, let $P = \langle p_1, \ldots, p_n \rangle$ and $Q = \langle q_1, \ldots, q_n \rangle$ be two sequences of points. We assume that P is translated so that its centroid $(\frac{1}{n}\sum_{k=1}^{n} p_k)$ is at

the origin. We also assume that Q is translated in the same way. For each point or vector x, let $(x)_i (i = 1, 2, 3)$ denote the i-th (X, Y, Z) coordinate value of x, and $\|x\|$ denote the length of x. Let

$$\text{RMSD}(P, Q, R, \mathbf{a}) = \sqrt{\frac{1}{n}\sum_{k=1}^{n} \|Rp_k + \mathbf{a} - q_k\|^2},$$

where R is a rotation matrix and $\mathbf{a}$ is a translation vector. Then, the *rmsd* value $d(P, Q)$ between P and Q is defined by $d(P, Q) = \min_{R,\mathbf{a}} d(P, Q, R, \mathbf{a})$. Schwartz [16] showed that $d(P, Q, R, \mathbf{a})$ is minimized when $\mathbf{a} = 0$ and

$$R = (A^t A)^{\frac{1}{2}} A^{-1},$$

where the matrix $A = (A_{ij})$ $i, j = 1, 2, 3$ is given by

$$A_{ij} = \sum_{k=1}^{n} (p_k)_i (q_k)_j,$$

where $A^{\frac{1}{2}} = B$ means $BB = A$, and $\mathbf{o}$ denotes the zero vector. Thus, $d(P, Q)$, R and $\mathbf{a}$ can be computed in $O(n)$ time [13].

We refer to Martin's ProFit package (standing for protein fitting system) [12] and write a program to calculate the *rmsd* between C-α atoms of paired protein backbones with C language. Fitting was performed using the McLachlan algorithm [13].

After locating the appropriate suggested points, the minimum bipartite matching algorithm is used to find the best matching between two sets to decide the best matching alignment, which is needed for the RMSD procedure [17]. Let $P' = T \circ P$, and Q being translated to Q' such that the mass center of Q' is located at the origin. We construct a weighed graph $G = (V, E)$ with V being labelled with points of P' and Q', and each (p, q) in E being weighted with the squared Euclidean (3D) distance; i.e., $w(p, q) = \|p, q\|^2$. We then solve the weighted *minimum bipartite matching* problem [6] to obtain the best matching of P' and Q'. By the matched pairing, we perturb and refine the final alignment to obtain a prosbably lower *rmsd*.

Isometric rotation transformation

According to Euler's rotation theorem [5], any rotation about the origin point can be described by using three angular parameters. The rotation is determined by 3 consecutive rotations with 3 *Euler angles* (α, β, γ). The first rotation is done by the angle α around the z-axis, the second is done by the angle β around the x-axis, and the third rotation is done by the angle γ around the z-axis. See [8] for more detailed discussions about the transformation.

Analog to Euler's setting, we can also consider the point of north-pole $\mathbf{n} = (0, 0, 1)^T$ on the unit sphere. After the rotation, R, say $\mathbf{n}$ is rotated to another point $\mathbf{p} = (x, y, z)^T$; i.e., $\mathbf{p} = R\mathbf{n}$. Let α denote the angle $\angle \mathbf{n}O\mathbf{p}$. Note that α determines the z-coordinate of $\mathbf{p}$. To determine x-coordinate and y-coordinate of $\mathbf{p}$, the point is rotated around the z-axis for the angle β on the unit sphere. Note that there are infinitely numbers of rotation that transform $\mathbf{n}$ to $\mathbf{p}$. The particular rotation R can be decided by rotating all other points around the vector $\mathbf{p}$ by the angle γ. It is not hard to verified that, in such a way, *any* rigid rotation transformation can be parameterized by the three-tuple (α, β, γ). Thus, we call a vector $\mathbf{p} = (x, y, z)^T$ on the surface of the unit sphere a *probe*. Note that the *movement* of each probe is started from the north-pole $(0, 0, 1)^T$ to other points in the sphere. The position of $\mathbf{p}$ is decided by the parameters (α, β), and exact rotation is fixed by the self-rotation angle γ.

As a result, we reduce the problem of finding the good rotation matrix to the new problem of finding a good 3-parameter setting. The rotation matrix is thus characterized by just adjusting the 3 uniformly distributed parameters.

Minimum bipartite matching

We use the minimum bipartite matching to find the best matching between two sets of points to decide the best matching for the *rmsd* procedure. We adopted the Munkres [14, 3, 2, 15] algorithm. The public available implementation is written with the Perl language. To improve the efficiency of computation, we implement the Munkres algorithm and write hundreds lines of C Codes.

2.1 Parametric adjustment with trigonometric series

In our previous work [17], the trigonometric series estimation method, the three parameters are assumed to be independent. We adjust the three parameters one by one and increase the power of the estimated function. The trigonometric series function is described as the following:

$$\begin{aligned} f(\theta) = C_1 &+ C_2 \cos \pi\theta + C_3 \sin \pi\theta \\ &+ C_4 \cos 2\pi\theta + C_5 \sin 2\pi\theta \\ &+ C_6 \cos 3\pi\theta + C_7 \sin 3\pi\theta + \ldots \\ &+ C_{2k} \cos k\pi\theta + C_{2k+1} \sin k\pi\theta. \end{aligned} \tag{1}$$

where $f(\theta)$ denote the corresponding value of *rmsd* with respect of one of the three parameters, (α, β, γ). The k usually reflects the numbers of local maximal points in the approximated curve.

2.2 Initialization by segment alignment

Comparing to the more sophisticated methods like CE or VAST, the main-vector initialization position [17] does have the advantage of saving valuable computation resources. Yet the initial orientation found by the main-vector method does not produce satisfactory final orientation even after the fine-tune procedures. The idea here is trying to find more suitable starting positions and still conserve enough computation time for later adjustment. Since the protein structure is just a chain sequence of atoms, we can subdivide the sequence and use the subsequence matching information to find the better alignment. Thus, the atom chains of a structure is divided into several (consecutive) segments.

Given a list of (consecutive) atoms obtained from the PDB file [1], one way of dividing protein chains of a structure is by using the secondary structural information of the given protein. That is, for the secondary structural partition method, the segments of structures is determined by partition the protein sequence by the secondary structural

information of the given protein. Another possible division scheme is obtained by slicing a fixed number of atoms of the given protein. Thus, for the fixed number partition method, the segments of structures is determined by partition the protein sequence by a fixed number of atoms of the given protein.

After the segments of structures is decided, the *segment alignment* uses the standard dynamic programming technique to obtain feasible pairings between segments by maintaining a suitable score table. The dynamic programming evaluation function is described as the following:

$$score(s, \lambda) = Ump \cdot \mid s \mid$$
$$score(\lambda, t) = Ump \cdot \mid t \mid$$
$$score(sx, ty) =$$
$$\min \begin{cases} \textsc{Rmsd}(L(s,t) \circ \textsc{Match}(x,y)) \cdot \ell \\ \quad + Ump \cdot (\mid sx \mid + \mid ty \mid -2\ell) \\ score(sx, t) + Ump \cdot \mid y \mid \\ score(s, ty) + Ump \cdot \mid x \mid \end{cases}$$

here λ denotes the empty list; s and t are two prefix segment lists. $L(s,t)$ is the alignment between segment lists s and y, and n_L denotes the number of atoms in L; $\ell = \mid L(s,t) \circ \textsc{Match}(x,y) \mid$.

The recurrence relation for evaluating the value *score* relies on three possible alignments between sx and ty. Here s and t are two prefix *segment lists*, and x and y are the two currently (last) considered segments. The first alignment, L is the pairing list from $L(s,t)$ merging with $\textsc{Match}(x,y)$ which stands for the match between segment x y. Since Rmsd() returns the average precalculated *rmsd* value, the number is multiplied by the number of matched pairs ℓ. However, if one can not find any match for an atom, a given punishment constant, Ump, must be added to encourage most atom be aligned with some atoms on the other sequence. Another possibility is the case of $score(sx, t)$; in that case, the segment y is not able to match with segment on the other list. Thus we need to add in the punishment values for all atoms of the y segment. The case of $score(s, ty)$ is also treated similarly.

3 Methodology

In our previous works, we have several initial setting methods including main vector, random, and segment alignment. In order to find the appropriate alignment list between the protein pair, the local refinement algorithm is used to refine the final alignment.

Nevertheless, the quality of the final results obtained from the local search and refinement methods is highly depended on the initial settings of starting search positions. Here we propose several possible settings of the initialization algorithms to examine the effectiveness of the local refinement algorithms given different initial settings obtained from the secondary structural information of the protein pair.

3.1 Segment alignment with secondary structural information

In biochemistry and structural biology, secondary structure is the general three-dimensional form of local segments of biopolymers such as proteins and nucleic acids. In proteins, the secondary structure is defined by patterns of hydrogen bonds between backbone amide and carboxyl groups (sidechain-mainchain and sidechain-sidechain hydrogen bonds are irrelevant), where the DSSP [19] definition of a hydrogen bond is used. The hydrogen bonding is correlated with other structural features, however, which has given rise to less formal definitions of secondary structure.

Since the protein structure is just a chain sequence of atoms, we can subdivide the sequence and use the subsequence matching information to find a better starting. Thus, the atom chains of a structure is divided into several (consecutive) segments. Although the 8-state DSSP code is already a simplification from the 20 amino acid residues present in a protein, the majority of secondary prediction methods simplify further to the three dominant states: *helix, sheet* and *coil* [19]. Early methods of secondary-structure prediction were based on the helix- or sheet-forming propensities of individual amino acids. Such methods were typically 60% accurate in predicting which of the three states (helix/sheet/coil) a residue adopts. A significant increase in accuracy (to nearly 80%) was made by exploiting multiple sequence alignment. The alpha-helix and beta-sheet conformations in globular protein structures, assigned by DSSP [19] and STRIDE [4] algorithms, were divided into regular and distorted fractions by considering a certain number of terminal residues in a given alpha-helix or beta-strand segment to be distorted.

In our previous works [10], we show that the dividing protein chains by slicing a fixed number of atoms of the given protein. In this paper, we examine other ways of dividing protein sequence by using the secondary structural information of the given protein.

Initialization setting by secondary structure

A good starting initial alignment is essential for the later trigonometric series estimation method to refine it into a final well adjusted alignment. Since the secondary structure of a protein reflects the underlying basic scheme of the tertiary structure, it is reasonable that the segment alignment algorithm proposed could incorporate the secondary structural information. Unfortunately, not every protein possess all three secondary structures (helix/sheet/coil). That is, some proteins don't have helix structure or sheet structure. To perform the suitable evaluation of whether the secondary structural information improves the final alignment, we selects those protein pairs that possess all three secondary structures. We also partition the protein atoms into segments by a fixed number of atoms after slicing the protein chains according to its secondary structure. In our previous work [10], it is shown that we would obtain satisfactory result when the fixed number is 3.

Based upon all these considerations, we use the following five possible different segmentation methods in testing the appropriateness of the methods in our experiments.

- HS: partition the (helix/sheet) subsequences of the protein into segments (of various length).
- CUT-3: partition the whole sequence of the protein into segments of length 3.
- HST-3: partition the (helix/sheet/coil) subsequences of the protein into segments of length 3.
- HS-3: partition the (helix/sheet) subsequences of the protein into segments of length 3.

```
CALC(n, m, r, t)
    Input: t[n, m], where t[i, j] is the time of j-th algorithm on the i-th pair of protein structure.
           r[n, m], where r[i, j] is the rmsd of j-th algorithm on the i-th pair of protein structure.
    Output: (r_A, t_A), where r_A is average rmsd of all structures and t_A is the average time spent by method A.
 1  A ← ∅        ▷ empty set
 2  while A ∉ {1, ..., m}
 3        do r_sum ← 0
 4           t_sum ← 0
 5           A ← NEXTSET(A)
 6           for i ← 1 to n
 7                 do r ← ∞
 8                    t ← 0
 9                    for each method j ∈ A
10                          do t ← t + t[i, j]
11                             r ← min(r, r[i, j])
12                    r_sum ← r_sum + r
13                    t_sum ← t_sum + t
14           r_A ← r_sum/n
15           t_A ← t_sum/n
16  return (t_A, r_A)'s        ▷ for all A ⊂ {1, ..., m}
```

NEXTSET(A) is the function that returns the next kind combination of A.

Figure 1: The input of the CALC(n, m, r, t) are $t[n, m]$ and $r[n, m]$, where j and i are the j-th algorithm and the i-th pair of protein structure. The output is (r_A, t_A), where r_A is average *rmsd* of all structures and t_A is the average time spent by method A.

- HST: partition the (helix/sheet/coil) subsequences of the protein into segments (of various length).

Supposed we are given two protein structures, namely, PA and PB. For the CUT-3 method, each protein structure is first segmented into a sequence of segments, each of length 3. Then the segment alignment algorithm is used by examining the *score* function, described in the previous section, to obtain the initial setting orientation, and then the algorithm proceeds by using the trigonometric series estimation method to adjust the parameters. The rest of other four methods are also conducted in a similar manner, the only differences are their different ways of partitioning the protein sequence into different sequence of segments.

3.2 Combining various secondary structural information

Other than adapting one single method in trying to segment the two sequences of the given protein pairs, it is possible that we might want to incorporate different combinations of these different segmentation methods. Note that the final *rmsd* quality and the generally concerned time resource spent on the method are two different measurement for the outcome of the algorithms. It is sometimes desirable to combine both features in one single formula. Here we propose the the following evaluation function for the appropriateness of the a combination of several methods. That is, supposed that there are m different methods, say method 1, 2, ..., m, for solving the same problem. We define the *goodness* of a combination of these m method, $A \subset \{1, 2, \ldots, m\}$, as

$$\mu(A) = -(K \log r_A + L \log t_A) \tag{2}$$

here K and L are two positive real numbers that adjust the (relative) stress of influence of the results between two measures r_A and t_A. Here r_A is the average *rmsd* value and t_A is the average time spent in trying out all methods of A. The calculation function CALC is adapted to compute the average *rmsd* value and average time, and is described as the following:

$$\text{CALC}(n, m, r, t) : \begin{cases} r_A = (\sum_{j=1}^{n} \min_{j \in A}\{r[i, j]\})/n \\ t_A = (\sum_{j=1}^{n} \sum_{j \in A}\{t[i, j]\})/n \end{cases} \tag{3}$$

here n denotes the number of the proteins; m is the number of methods of segmenting the protein sequence. Note that $r[i, j]$'s is the array of *rmsd* value of data instance i obtained by method $j \in A$. The minimum *rmsd* value obtained from all $j \in A$ kinds of segmentations is denoted by r_A, while the total time spent is denoted by t_A. The algorithms is illustrated in Figure 1.

The rationales behind the formula are the following. Given two combinations $A, B \subset \{1, 2, \ldots, m\}$ with different *rmsd* and time spent, we have the the following:

$$\begin{aligned} \phi(A, B) &= \mu(A) - \mu(B) \\ &= -(K \log r_A + L \log t_A) - (-K \log r_B - L \log t_B) \\ &= K \log(r_B/r_A) + L \log(t_B/t_A) \\ &= \log\left((r_B/r_A)^K \cdot (t_B/t_A)^L\right) \end{aligned} \tag{4}$$

Note that the quality (goodness) of an algorithm A is referred by justifying whether a method obtains a smaller *rmsd*, r_A or spends less time spent, t_A. Thus, the *relative measurement* of r_B/r_A and t_B/t_A account for the superiority of one method A comparing to the other method B; actually, the larger value implies how better the method A is, comparing to method B. Thus $\phi(A, B)$, or the log value of $(r_B/r_A)^K(t_B/t_A)^L$, stands for the superiority of method A over method B. Note that a positive $\phi(A, B)$ suggests method A is better than method B, while a negative $\phi(A, B)$ suggests otherwise.

That is, we conclude that the goodness measurement $\mu(A)$ is an appropriate way of evaluating the combined

method A. Furthermore, we can say method A is better than method B whenever $\mu(A) > \mu(B)$; meaning $\mu(\cdot)$ is a reasonable scoring function for evaluating these different combinations of methods. Actually, we have the following:

$$\begin{aligned} Score(A,B) &= e^{[\mu(A)-\mu(B)]} \\ &= e^{\phi(A,B)} = (r_B/r_A)^K (t_B/t_A)^L \end{aligned} \tag{5}$$

Selecting K and L

In previous discussion, we conclude that, whenever $\mu(A) > \mu(B)$ then the method A is considered better than method B. Actually, by the definition of $\mu(\cdot)$ function, it is easily concluded that

Proposition 1 *If $t_A < t_B$ and $r_A < r_B$, then method A is better than method B since $\mu(A)$ is always greater than $\mu(B)$.*

On the other hand, suppose that we are given the case that $t_A < t_B$ but $r_A > r_B$. Now, the different setting of K and L affects the final decisive $\mu(\cdot)$ values. Actually, we have a critical boundary ρ, such that, when $K/L < \rho$, we have $\mu(A) > \mu(B)$; however, when $K/L > \rho$, we have $\mu(B) > \mu(A)$. How is the critical boundary ρ determined? The K/L ratio actually determines the *quality/time* preference of a user chooses.

$$\rho = K/L \tag{6}$$

We observe that the *critical boundary* of two methods A and B, $\psi(A,B)$, is defined by the following:

$$\psi(A,B) = \frac{\log(t_B/t_A)}{\log(r_A/r_B)} \tag{7}$$

Lemma 1 (critical boundary of K/L) *Suppose that $t_A < t_B$ but $r_A > r_B$. It follows that method A is better than method B if and only if $(K/L) < \psi(A,B)$.*

Proof. By definition, the fact that method A is better than method B implies $\mu(A) > \mu(B)$. Thus $K\log r_A + L\log t_A < K\log r_B + L\log t_B$. Then $K[\log r_A - \log r_B] < L[\log t_B - \log t_A] \Leftrightarrow K/L < (\log t_B/t_A)/(\log r_A/r_B) = \psi(A,B)$. □

Suppose that we need to choose two different combinations A or B. It may happen that $t_A < t_B$ and $r_A < r_B$ then method A is always superior to method B, since $\mu(A) > \mu(B)$. In that case, the method B will never be considered as a good choice. On the other hand, if no other method A out there that guarantees $t_A < t_B$ and $r_A < r_B$. Can we conclude method B will someday be considered as a good choice by selecting an appropriate K/L ratio? The answer is probably not. Since it may happen that a method B is sometimes *clouded* by the other methods A and C as the following observation shows; the situation is also illustrated in Figure 2. In that case, the method B will never be considered as an appropriate method whatever the setting of K/L is.

Corollary 1 *Suppose that $r_A > r_B > r_C$ and $t_A < t_B < t_C$. It follows that the method B is never better than A or C if and only if $\psi(A,B) > \psi(B,C)$. In other words, method B is clouded by method A and C, and thus will never be selected as a candidate method as shown in Figure 3.*

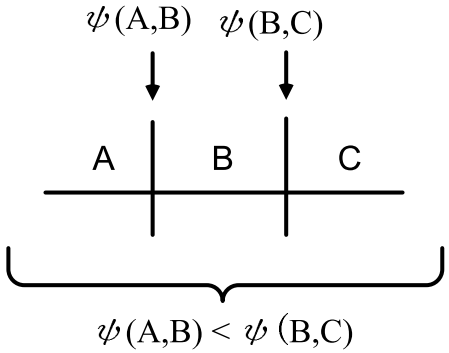


Figure 2: $\psi(A,B) < \psi(B,C)$, the usual case. Note that method B is appropriate if and only if $\psi(A,B) < K/L < \psi(B,C)$..

Proof. Note that when $\psi(B,C) < \psi(A,B)$ we have that method B is never chosen because either $(K/L) < \psi(A,B)$ then method A is preferred than method B; otherwise, $(K/L) > \psi(A,B) > \psi(B,C)$. Then method C is now preferred than method B. That is, method B is never the best choice since it is worse than method A or method C. □

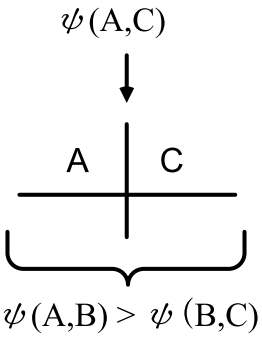


Figure 3:

Suppose that there are k different methods under consideration. Let M_1 be the fastest method and M_k be the method rendering the best quality (and thus the most time-consuming). According to Lemma 1, we must have $t_1 < t_2 < \cdots < t_k$ and $r_1 > r_2 > \cdots > r_k$; otherwise, some of these methods would never be under consideration. Furthermore, in order to discriminate the suitability of those methods, we define the *CB-ratio*, or the *critical boundary ratio*, of method M_i, denoted by $\mathcal{L}[M_i]$, as the following:

$$\mathcal{L}[M_i] = \psi(M_i, M_{i+1})/\psi(M_{i-1}, M_i) \tag{8}$$

and the situation is illustrated in Figure 4. Note that, if

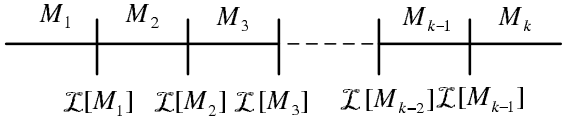


Figure 4:

a user's q/t-ratio, K/L, is in the range of $\psi(M_{i-1}, M_i)$ to $\psi(M_i, M_{i+1})$, then method M_i is preferred. The widely

ranged popularity of $\mathcal{L}[M_i]$ to $\mathcal{L}[M_{i+1}]$ reflects how one user would choose this method before the q/t-ratio is factored by a sufficiently large amount, as the situation illustrated in Figure 2. Therefore, it would seem that $\mathcal{L}[M_i]$ is a measurement reflecting the *popularity* of method M_i.

4 Experimental Results

In this section, we introduce the target of experimental data set first. Then we show the difference with dividing protein chains of a structure depends on the different secondary structures of the given protein. Finally, we show the Web enable user can use our system through the network.

4.1 Data Set

We choose the PDB for our experimental sample source, and we randomly pick 200 protein structures in the PDB database as our experimental subjects by the uniform distribution sampling out of totally 27,835 protein structures as of 2007. For each chosen protein structures we randomly choose 30 structure alignments listed on the database of VAST as the tested targets. We use the term, P, to stand for one of the 200 randomly picked protein structures, and we use Q to stand for one of the 30 neighbors of each P. Note that P and Q include all un-aligned and aligned atoms. We use the term, PA, to stand for the aligned atoms of P by VAST, and we use PB to stand for one of the 30 neighbors of each PA. Totally, there are 6,000 protein pairings tested by our previous experiment. The distribution of them is shown in Figure 6. The Q stand for one of the 30 neighbors of each P, and different P may use the same protein Q. At last we pick 200 protein structures in the PDB database for PA, and choose 3352 different protein structures in the PDB database for PB.

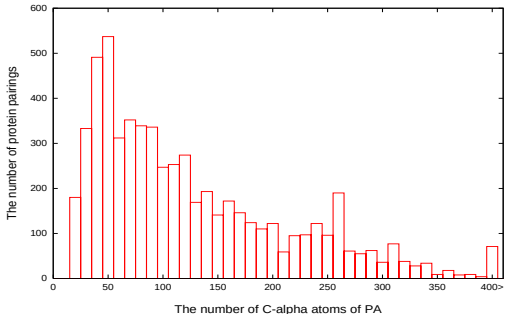


Figure 6: The distribution of the 200 randomly picked protein structures in PDB and their 30 neighbor structures. The total number of protein pairs is 6,000.

Unfortunately, not every protein possess all three secondary structures (helix/sheet/coil). In our experimental, there are many proteins dose not have the helix structure. There have some experiment can not work if we only refer to helix structure. There are some proteins don't have helix structure or sheet structure but must of protein have at least one of the two structure. So we choose those five different ways: HS, CUT-3, HST-3, HS-3 and HST. In our experiment, we reject the data that some special protein can't make operation. For example the protein 1I97 does not have any secondary structure in PDB file. We must remove those data from our data set. Then we remove 154 protein pairings for 6000 protein pairings.

4.2 Parameter Adjustment

There are five different initial rotations in the following algorithms CUT-3, HS (helix+sheet), HS-3 (helix+sheet+CUT-3), HST (helix+sheet) and HST-3 (helix+sheet+CUT-3). The result shown in Table 2 is to execute the five divided ways and VAST. The results time shown in Table 3. We don't have the VAST operation's time, so we don't have express in the table. In our experiment, we use five bits to encode the number of the methods set show in Table 1. For example the subset of methods {HST, HST-3, HS} that we use $(10101)_2$ to represent. We also can call that $(21)_{10}$.

HST	HS-3	HST-3	CUT-3	HS	number
0	0	0	0	1	1
0	0	0	1	0	2
0	0	0	1	1	3
⋮	⋮	⋮	⋮	⋮	⋮
1	1	1	1	1	31

Table 1: We use the five-bit string to represent the set of combination of these five methods, HS, CUT-3, HST-3, HS-3 AND HST.

In our experimental results and shown *CB-ratio* in Figure 5. When the value K/L is small, it's emphasize with time. If we want to care about quality of *rmsd* more, then we can choose a big value of K/L. We want to use the minimum of time, then we choose the value K/L small than 0.5 and it will choose the method $(1)_{10}$(HS). On the other hand we want to get nice quality of *rmsd*, it is obvious to see that choose all methods (method $(31)_{10}$) can get the minimum *rmsd* in our five methods. In Figure 5, we can see the *critical boundary*. The ratio of $\psi(1,2)$ to $\psi(2,3)$ is 269.82. It is bigger than other ratio. When user choose K/L in method $(2)_{10}$ area. User want to care about quality of *rmsd* more. He must more multiple of K/L to go to next area then other methods. On the other hand the method $(15)_{10}$ have the ratio of $\psi(7,15)$ to $\psi(15,31)$ very small. That means method $(15)_{10}$ easy to change to next method when change K/L ratio.

Based on the result, the average of *rmsd* is the best by CUT-3. But it spends the most of the time. In some case of the protein doesn't have the best result of *rmsd* in CUT-3, but the smallest *rmsd* will in the other divide ways. If we choose the method 3 which include two divide ways CUT-3 and HS. It will spends more time but get the *rmsd* smaller than method 1 or method 2. However method 3 have smaller *rmsd* then method 1 or method 2 but it spends time is summation of method 1 and method 2. So if we choose a good K and L, every method has the chance to be elected. But a method have both *rmsd* and time bigger than the other, it will never have the chance to be elected. In our experiment, by the Lemma 1 we first delete methods which are never have the chance to be elected. Then the remainder methods are following : method $(1)_{10}$, method

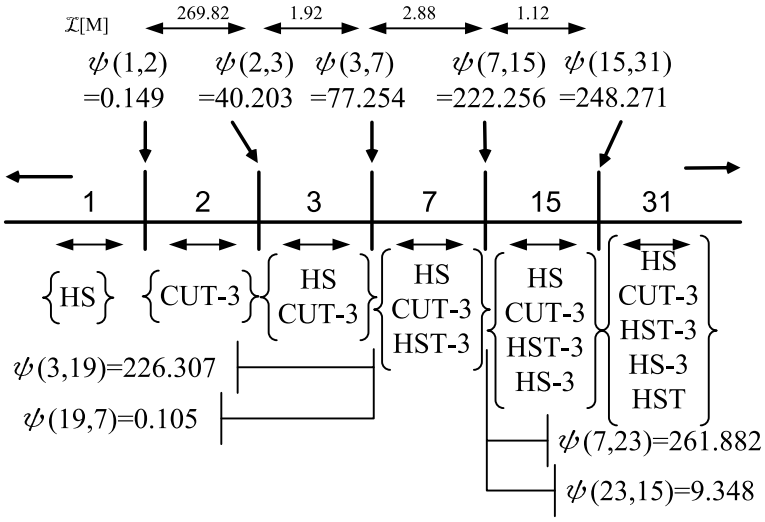


Figure 5: When the value K/L is small, it's emphasize with time. If we want to care about quality of *rmsd* more, then we can choose a big value of K/L. Note in this figure we can detect that method $(23)_{10}$ and method $(19)_{10}$ will not be selected forever.

$(2)_{10}$, method $(3)_{10}$, method $(7)_{10}$, method $(15)_{10}$, method $(19)_{10}$, method $(23)_{10}$ and method $(31)_{10}$. We try to find when each of methods have be selected. By the Corollary 1, we find the method $(23)_{10}$ and the method $(19)_{10}$ can't be elected for ever. In this case of method $(19)_{10}$, the method $(19)_{10}$ spends time less than method $(7)_{10}$ but quality is bad than method $(7)_{10}$. However, it spends time large than method $(3)_{10}$ and quality is better than method $(3)_{10}$. By the Corollary 1 $\psi(3,19) > \psi(19,7)$ so the method $(19)_{10}$ will never be choice.

CUT-3 spends much time then others initial method. It is not a good idea by using CUT-3 initial method if the number of C-α is very large and we show the average time is measured by t seconds per pair of structure in Table 3. In case number of 241-270 C-α atom of PA, CUT-3 spends more 27.79 seconds than HS, but the number of protein pairings are 370. It spends more time about 171 minutes. When K is more and more bigger, the *rmsd* is more important and we can select more different divide ways. On the other hand the value of L care about time effectiveness. When K/L from 78 to 222 will always select method $(7)_{10}$, but method $(23)_{10}$ and method $(19)_{10}$ never be selected. Note that CUT-3 is considered a good method because of its q/t ratio is clearly much larger than other methods in the analysis. We show the average time and *rmsd* in the Figure 7 and Figure 8. Most of the our methods result is better than VAST. If use the method $(31)_{10}$, the quality of average *rmsd* is 2.54, and the average *rmsd* after VAST method is 3.11. Our method improves the result of the VAST about 18.3%.

4.3 Web Accessibility

We make a web for user who is interesting in our protein structures comparison system at `http://bioinfo.cs.pu.edu.tw/palig.html`. The user of our web service usually provides two protein PDB IDs or just directly input the protein structure in PDB file format. The quality/time preference values of K and L can be provided by the web user as well. Besides, user also can select several algorithms from five candidate

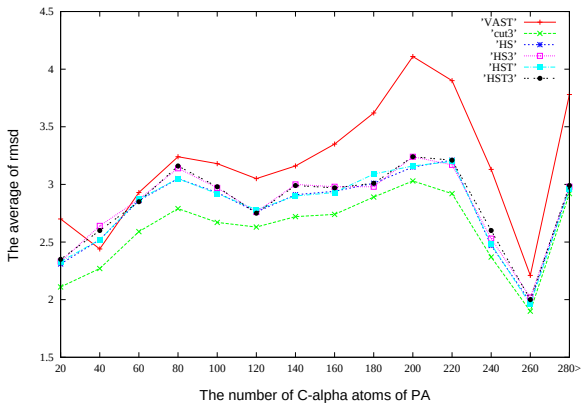


Figure 7: The average of rmsd value for VAST, CUT-3, HS, HS-3, HST, HST-3.

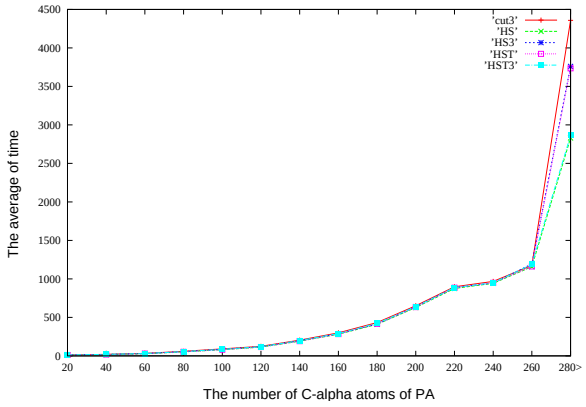


Figure 8: The average of time value for VAST, CUT-3, HS, HS-3, HST, HST-3.

The number of C-α atom of PA	The numbers of protein pairings	The average *rmsd* after VAST	The average *rmsd* by CUT-3	The average *rmsd* by HS	The average *rmsd* by HS-3	The average *rmsd* by HST	The average *rmsd* by HST-3
12-30	392	2.56	2.25	2.37	2.41	2.39	2.41
31-60	1334	2.74	2.48	2.75	2.81	2.75	2.78
61-90	1000	3.25	2.74	3.02	3.10	3.03	3.10
91-120	753	3.05	2.62	2.76	2.78	2.76	2.78
121-150	504	3.23	2.67	2.90	2.95	2.90	2.95
151-180	430	3.51	2.83	2.93	2.96	2.97	2.97
181-210	300	4.13	3.09	3.24	3.30	3.26	3.31
211-240	314	3.30	2.54	2.68	2.72	2.68	2.76
241-270	370	2.39	1.99	2.04	2.07	2.03	2.06
271-300	137	3.94	2.95	3.02	3.03	3.02	3.02
301-330	154	3.65	2.81	2.85	2.95	2.86	2.96
331-360	67	4.40	3.20	3.22	3.26	3.22	3.26
361-390	21	3.29	2.72	2.73	2.80	2.72	2.79
390-1200	70	3.87	2.97	3.00	3.01	3.00	3.01
total	5846	3.11	2.61	2.79	2.84	2.80	2.84

Table 2: The result is to execute the algorithm of five initial method and VAST. The unit of *rmsd* values is measured by Angstrom($\mathring{A} = 10^{-8}cm.$).

The number of C-α atom of PA	The numbers of protein pairings	The average time by CUT-3	The average time by HS	The average time by HS-3	The average time by HST	The average time by HST-3
12-30	392	15.14	13.95	14.57	14.05	14.58
31-60	1334	28.99	25.25	27.11	25.58	26.92
61-90	1000	69.69	60.71	65.40	61.42	64.80
91-120	753	116.49	106.92	111.84	107.48	111.30
121-150	504	226.94	211.59	219.39	213.63	219.02
151-180	430	403.26	380.73	392.18	388.70	391.96
181-210	300	691.00	669.32	685.58	677.78	680.98
211-240	314	953.88	931.94	942.83	943.48	937.30
241-270	370	1262.55	1234.76	1267.79	1243.08	1260.22
271-300	137	2064.83	1985.43	2038.33	2006.54	2019.59
301-330	154	2822.55	2748.56	2804.30	2781.45	2786.52
331-360	67	4827.05	4697.70	4759.82	4727.18	4763.19
361-390	21	4692.83	4795.94	4888.26	4808.42	4863.27
390-1200	70	41394.66	40218.13	40574.29	40518.91	40263.36

Table 3: The average time is measured by t seconds per pair of structure.

The initial method method	The average *rmsd* after initial method	The average *rmsd* after one MBM	The average *rmsd* after trigonometric
VAST	3.11	-	-
CUT-3	4.33	2.78 (10.61%)	2.61 (16.08%)
HS	5.04	3.26 (-4.82%)	2.80 (9.96%)
HS-3	5.09	3.32 (-6.75%)	2.85 (8.36%)
HST	5.13	3.26 (-4.82%)	2.81 (9.65%)
HST-3	5.18	3.34 (-7.40%)	2.85 (8.36%)
CUT-3 + HS	-	-	2.57 (17.36%)
CUT-3 + HS + HST-3	-	-	2.56 (17.68%)

Table 4: The result is to execute the algorithm of trigonometric series with initial alignment of CUT-3, HS, HS-3, HST and HST-3.

algorithms. Our web service will search and obtain the corresponding PDB data from the Protein data bank database [1] and perform the desired protein structure alignment/comparison using the chosen set of algorithms. To avoid the time delay, our web service also provides user access keys for user to check the result later on after they submit the desired query. User can come back and check the result after operation finished by our local server; parts of our web entry interface are shown in Figure 9.

5 Concluding Remarks

In this paper, we propose methods of combining different segmentation methods in trying to improve the quality of the final *rmsd* value between a protein structure pair by finding better alignment list. As it is shown by the experiment results, our method improves the alignment computed by the VAST package by an averaged improvement ratios about 18.3%.

The best quality is obtained by combining five different initial segmentation methods CUT-3, HS, HS-3, HST, and HST-3. These settings are later adjusted by local refinement algorithm with trigonometric series estimation. The combined method spends the most time comparing to other proposed combined methods. It is shown that the combined method by HS and CUT-3 spend average time about 1832 seconds but improvement ratios about 17.7% although the averaged running time is about 39.8% less. For the discussion of our experimental results, CUT-3 has large widely *CB-ratio* than others. That is very interesting about unit, *rmsd* value is usually smaller then 6 but the value of time is very large. The *rmsd* value improvement ratio is easy larger then time improvement ratio. Even CUT-3 spends more time but the *rmsd* value compared to time improvement ratio is large. We can know that is popularity for user to choose. In our previous work [17], the MBM and trigonometric series estimation can help us to improve the result of our initial alignment. The result is to execute the algorithm of trigonometric series with initial alignment of CUT-3, HS, HS-3, HST and HST-3 and show in Table 4.

Here in this paper we proposed ways of incorporating both measurement of the time resource as well as the final quality, namely *rmsd* that we are more care about. We show that it is possible to adjust the parameters to get the result that are sufficiently satisfactory and yet spent less time resources. Furthermore, some of our preliminary result is accessible through the our web interface to provide molecular biologists and other bioinformatic researchers the use of our service.

References

[1] H. M. Berman, J. Westbrook, Z. Feng, G. Gilliland, T. N. Bhat, H. Weissig, I. N. Shindyalov, and P. E. Bourne. The protein data bank. *Nucleic Acids Research*, 28:235–242, 2000.

[2] F. Bourgeois and J. C. Lassalle. Algorithm 415: Algorithm for the assignment problem (rectangular matrices). In *Communications of the ACM*, volume 14, pages 805 – 806, New York, NY, 1971. USA.

[3] F. Bourgeois and J. C. Lassalle. An extension of the munkres algorithm for the assignment problem to rectangular matrices. In *Communications of the ACM*, volume 14, pages 802 – 804, New York, NY, 1971. USA.

[4] Frishman D and Argos P. Knowledge-based protein secondary structure assignment. In *Proteins: Structure, Function, and Genetics*, pages 566–579, 1995.

[5] L. Euler. Formulae generales pro trandlatione quacunque corporum rigidorum. *Novi Acad. Sci. Petrop.*, 20:189–207, 1775.

[6] Z. Galil. Efficient algorithms for finding maximum matching in graphs. *ACM Computing Surveys*, 18:1:23–38, 1986.

[7] J. F. Gibrat, T. Madej, and S. H. Bryant. Surprising similarities in structure comparison. *Curr Opin Struct Biol*, 6(3):377–385, 1996 Jun.

[8] A. Gray. A treatise on gyrostatics and rotational motion. MacMillan,London, 1918.

[9] L. Holm and C. Sander. Touring protein fold space with DALI/FSSP. *Nucleic Acids Res.*, 26:316–319, 1998.

[10] W. D. Jiang, Y. L. Lin, and H. S. Shin. Local rfinement algorithms for protein structure comparison and alignment. In *Proceedings of the International computer symposium 2008*, pages 9–14, 2008.

[11] Zhang T. Zhang H. Shen S. Ruan J. Kurgan, L.A. Secondary structure-based assignment of the protein structural classes. In *Amino Acids*, pages 551–564, 2008.

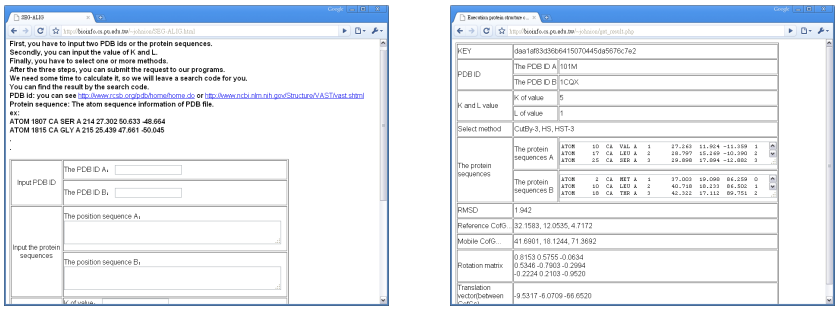
[12] A. C. R. Martin. `http://www.bioinf.org.uk/software/profit/`.

[13] A. D. McLachlan. Rapid comparison of protein structures. *Acta Cryst*, A38:871–873, 1982.

[14] J. Munkres. Algorithms for the assignment and transportation problems. *Journal of the Society for Industrial and Applied Mathematics*, 5:32–38, 1957.

[15] R. A. Pilgrim. `http://csclab.murraystate.edu/bob.pilgrim/445/munkres.html`.

[16] J. T. Schwartz and M. Sharir. Identification of partially obscured objects in two and three dimensions by matching noisy characteristic curves. *Int. J. Robotics Research*, 6:29–44, 1987.



(a) (b)

Figure 9: (a)The web window of input and user menu. User submits can get a access key. After the system obtain all analyzed results, user can get the result later on through the use of access key. (b)User submit the access key will see the result.

[17] H. S. Shin, Y. L. Lin, and W. D. Jiang. Protein structures alignment algorithms by parametric searching with trigonometric series. In *Proceedings of the 25th Workshop on Combinatorial Mathematics and Computation Theory*, pages 44–54, Hsinchu, Taiwan, 2008.

[18] I. N. Shindyalov and P. E. Bourne. Protein structure alignment by incremental combinatorial extension (CE) of the optimal path. *Protein Eng.*, 11:739–747, 1998.

[19] Kabsch W. and Sander C. Dictionary of protein secondary structure: pattern recognition of hydrogen-bonded and geometrical features. In *Biopolymers*, pages 2577–2637, 1983.